LAPORAN PROYEK DATA MINING

KLASIFIKASI GENOTIPE TIKUS BERDASARKAN PROFIL EKSPRESI PROTEIN MENGGUNAKAN K-NEAREST NEIGHBOR, GAUSSIAN NAIVE BAYES, DAN DECISION TREE

Disusun oleh:

Muhammad Aqil Mubarak (250810701100003)

JURUSAN INFORMATIKA

FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM

UNIVERSITAS SYIAH KUALA

BANDA ACEH

2026

ABSTRAK

Down Syndrome merupakan kelainan genetik akibat trisomi kromosom 21 yang berdampak pada fungsi kognitif dan plastisitas sinaptik. Penelitian ini mengklasifikasikan genotipe tikus, yaitu Control dan Ts65Dn, berdasarkan profil ekspresi 77 protein pada korteks serebral menggunakan dataset Mice Protein Expression yang terdiri atas 1.095 pengukuran dari 73 ekor tikus. Tahap praproses meliputi imputasi median untuk nilai hilang, pengodean target biner, normalisasi Min-Max, dan pembagian data latih dan uji dengan rasio 70:30 secara terstratifikasi. Tiga metode dibandingkan, yaitu K-Nearest Neighbor (K-NN), Gaussian Naive Bayes (GNB), dan Decision Tree berbasis entropi yang dioptimasi melalui GridSearchCV. Pada data uji, K-NN dengan k = 3 memperoleh akurasi, F1-score, dan AUC sebesar 1,0000, Decision Tree memperoleh akurasi 89,67% dengan AUC 0,9256, sedangkan GNB memperoleh akurasi 79,33% dengan AUC 0,8967. Analisis lanjutan menunjukkan bahwa skor sempurna K-NN tidak mencerminkan kemampuan generalisasi. Seluruh 73 tikus yang muncul di data uji juga terdapat di data latih, dan tetangga terdekat dari setiap sampel uji selalu berasal dari tikus yang sama. Ketika evaluasi diulang dengan pembagian berbasis individu tikus, akurasi K-NN turun menjadi 71,72%, sementara Decision Tree masih mencapai 80,46%. Hasil K-NN pada skema pembagian per baris dengan demikian bersifat bias optimis dan menunjukkan overfitting terhadap identitas individu. Decision Tree dinilai paling sesuai untuk konteks biomedis karena performanya lebih stabil dan aturan keputusannya dapat ditafsirkan, dengan APP_N sebagai protein yang paling berpengaruh.

Kata kunci: *Down Syndrome*, Ts65Dn, ekspresi protein, *K-Nearest Neighbor*, *Gaussian Naive Bayes*, *Decision Tree*, kebocoran data

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Allah SWT atas rahmat dan karunia-Nya sehingga laporan proyek yang berjudul "Klasifikasi Genotipe Tikus Berdasarkan Profil Ekspresi Protein Menggunakan *K-Nearest Neighbor*, *Gaussian Naive Bayes*, dan *Decision Tree*" ini dapat diselesaikan. Laporan ini disusun untuk memenuhi tugas Ujian Tengah Semester mata kuliah Data Mining pada Jurusan Informatika, Fakultas Matematika dan Ilmu Pengetahuan Alam, Universitas Syiah Kuala.

Proyek ini tidak hanya membandingkan angka performa tiga algoritma klasifikasi, tetapi juga menelusuri alasan di balik angka tersebut. Salah satu temuan yang mendapat perhatian khusus adalah skor sempurna yang diperoleh K-NN. Penulis berusaha menjelaskan secara jujur bahwa skor tersebut lebih banyak dipengaruhi oleh struktur dataset daripada oleh keunggulan algoritma, sehingga pembaca dapat menilai hasil penelitian ini secara proporsional.

Penulis mengucapkan terima kasih kepada dosen pengampu mata kuliah Data Mining atas bimbingan dan materi yang diberikan selama perkuliahan. Penulis menyadari bahwa laporan ini masih memiliki kekurangan, sehingga kritik dan saran yang membangun sangat diharapkan untuk perbaikan di masa mendatang.

Banda Aceh, 16 Oktober 2026

Penulis

DAFTAR ISI

ABSTRAK ..... **i**

KATA PENGANTAR ..... **ii**

DAFTAR TABEL ..... **vi**

DAFTAR GAMBAR ..... **vii**

BAB I PENDAHULUAN **1**

1.1 Latar Belakang Masalah 1

1.2 Deskripsi Dataset 2

1.3 Tujuan Proyek 3

BAB II METODOLOGI **4**

2.1 Tahap Praproses Data 4

2.1.1 Penanganan Nilai Hilang 4

2.1.2 Pemisahan Fitur dan Pengodean Target 4

2.1.3 Normalisasi Min-Max 4

2.1.4 Pembagian Data Latih dan Data Uji 5

2.2 Deskripsi Tiga Metode Klasifikasi 5

2.2.1 K-Nearest Neighbor (K-NN) 5

2.2.2 Gaussian Naive Bayes (GNB) 6

2.2.3 Decision Tree 7

2.3 Metrik Evaluasi 8

2.4 Validasi Tambahan Berbasis Individu Tikus 9

BAB III IMPLEMENTASI KODE & REPRODUCIBLE **10**

3.1 Lingkungan Komputasi 10

3.2 Struktur Proyek 10

3.3 Potongan Kode Utama 11

3.3.1 Praproses Data 11

3.3.2 K-Nearest Neighbor 11

3.3.3 Gaussian Naive Bayes....12
3.3.4 Decision Tree dengan GridSearchCV....12
3.3.5 Kurva ROC dan AUC....12
3.4 Langkah Reproduksi....12
BAB IV HASIL & PEMBAHASAN....14
4.1 EDA (Exploratory Data Analysis)....14
4.1.1 Ringkasan Dataset....14
4.1.2 Profil Missing Values dan Justifikasi Imputasi Median....15
4.1.3 Rekayasa Fitur dan Pembagian Data....16
4.1.4 Urgensi Normalisasi Min-Max untuk K-NN....16
4.2 Klasifikasi dengan K-NN (K-Nearest Neighbor)....17
4.2.1 Hasil Variasi Nilai k....17
4.2.2 Pemilihan Nilai k Terbaik....18
4.2.3 Klarifikasi Skor Sempurna K-NN dan Penentuan Bias....18
4.3 Klasifikasi dengan Gaussian Naive Bayes....21
4.3.1 Hasil Evaluasi....21
4.3.2 Diskusi Asumsi Conditional Independence....22
4.4 Klasifikasi dengan Decision Tree....23
4.4.1 Hasil Hyperparameter Tuning dan Evaluasi....23
4.4.2 Interpretasi Aturan Pohon Keputusan....24
4.4.3 Sepuluh Protein Terpenting dan Kaitan Biologisnya....25
4.5 Perbandingan dan Evaluasi Model....26
4.5.1 Tabel Perbandingan Kinerja....26
4.5.2 Karakteristik Kurva ROC dan Nilai AUC....27
4.5.3 Analisis Kesesuaian Metode untuk Dataset Mice Protein Expression....28
4.5.4 Analisis Komparatif Kelebihan dan Kekurangan Metode....29
BAB V KESIMPULAN DAN REKOMENDASI....30

5.1 Kesimpulan 30
5.2 Rekomendasi 30
REFERENSI 32
LAMPIRAN 34
Lampiran A. Aturan Lengkap Pohon Keputusan 34
Lampiran B. Kode Validasi Berbasis Individu Tikus 35
Lampiran C. Source Code 36

DAFTAR TABEL

Tabel 1.1 Ringkasan karakteristik dataset 2
Tabel 1.2 Distribusi atribut kategorikal 3
Tabel 3.1 Pustaka dan versi yang digunakan 10
Tabel 3.2 Struktur berkas proyek 10
Tabel 3.3 Urutan isi notebook main.ipynb 11
Tabel 4.1 Statistik deskriptif sepuluh protein utama sebelum praproses 14
Tabel 4.2 Rata-rata ekspresi protein utama berdasarkan genotipe 15
Tabel 4.3 Sebaran nilai hilang pada kolom protein 15
Tabel 4.4 Komposisi data latih dan data uji 16
Tabel 4.5 Performa K-NN pada data uji untuk berbagai nilai k 17
Tabel 4.6 Proporsi tetangga dari tikus yang sama dan akurasi K-NN pada dua skema pembagian data 19
Tabel 4.7 Performa Gaussian Naive Bayes pada data uji 21
Tabel 4.8 Matriks konfusi Gaussian Naive Bayes 21
Tabel 4.9 Hasil hyperparameter tuning Decision Tree 23
Tabel 4.10 Performa Decision Tree pada data uji 23
Tabel 4.11 Sepuluh protein terpenting dan kaitan biologisnya 25
Tabel 4.12 Perbandingan performa ketiga metode pada data uji (pembagian per baris) 26
Tabel 4.13 Perbandingan performa pada validasi berbasis individu tikus (StratifiedGroupKFold 5 lipatan) 26
Tabel 4.14 Analisis kesesuaian metode terhadap karakteristik dataset 28
Tabel 4.15 Kelebihan dan kekurangan ketiga metode 29

DAFTAR GAMBAR

Gambar 4.1 Pengaruh nilai k terhadap performa K-NN 17
Gambar 4.2 Matriks konfusi Gaussian Naive Bayes..... 22
Gambar 4.3 Struktur pohon keputusan hasil tuning (kedalaman 6, 27 daun) 24
Gambar 4.4 Sepuluh protein terpenting menurut Decision Tree 25
Gambar 4.5 Perbandingan kurva ROC ketiga model..... 27

BAB I
PENDAHULUAN

1.1 Latar Belakang Masalah

Down Syndrome (DS) adalah kelainan genetik yang terjadi karena adanya salinan tambahan kromosom 21 pada manusia. Kelebihan materi genetik ini menyebabkan gen-gen pada kromosom 21 diekspresikan lebih banyak dari kondisi normal, sebuah fenomena yang dikenal sebagai efek dosis gen (*gene dosage effect*). Ketidakseimbangan tersebut memengaruhi banyak jalur pensinyalan di otak, terutama yang berkaitan dengan pembentukan dan penguatan sinapsis, sehingga penderita DS umumnya mengalami hambatan dalam belajar dan mengingat (Antonarakis dkk., 2020).

Penelitian terhadap mekanisme biologis DS banyak memanfaatkan hewan model. Tikus galur Ts65Dn merupakan model yang paling sering digunakan karena membawa salinan tambahan segmen kromosom 16 tikus yang memuat banyak gen ortolog dari kromosom 21 manusia. Tikus ini memperlihatkan gangguan belajar kontekstual yang mirip dengan kondisi pada manusia (Contestabile dkk., 2017; Herault dkk., 2017). Higuera, Gardiner, dan Cios (2015) kemudian mengukur tingkat ekspresi 77 protein pada fraksi nuklir korteks serebral tikus Control dan Ts65Dn untuk mengetahui protein mana yang berperan dalam proses belajar.

Dari sisi *data mining*, data ekspresi protein memiliki sejumlah tantangan. Jumlah fiturnya cukup banyak, rentang nilai antarprotein sangat berbeda, sebagian nilai hilang, dan banyak protein saling berkorelasi karena berada pada jalur pensinyalan yang sama. Karakteristik ini membuat pemilihan metode klasifikasi tidak bisa dilakukan sembarangan. Algoritma berbasis jarak seperti K-NN sangat bergantung pada skala data, algoritma probabilistik seperti *Gaussian Naive Bayes* mengandalkan asumsi independensi antarfitur, sedangkan *Decision Tree* mampu menghasilkan aturan yang mudah dibaca tetapi rentan terhadap *overfitting*.

Selain pemilihan algoritma, cara mengevaluasi model juga perlu diperhatikan. Dataset ini berisi 15 pengukuran berulang untuk setiap ekor tikus, sehingga baris-baris data tidak sepenuhnya saling bebas. Jika struktur ini diabaikan, model dapat terlihat sangat akurat padahal sebenarnya hanya mengenali tikus yang sudah pernah dilihat. Oleh karena itu, penelitian ini juga menelaah apakah performa yang diperoleh benar-benar mencerminkan kemampuan model untuk mengenali genotipe pada tikus baru.

Berdasarkan uraian tersebut, rumusan masalah dalam penelitian ini adalah sebagai berikut.

1. Bagaimana tahapan praproses yang tepat untuk data ekspresi 77 protein yang memiliki nilai hilang, pencilan, dan perbedaan skala?
2. Bagaimana performa K-NN, *Gaussian Naive Bayes*, dan *Decision Tree* dalam membedakan genotipe Control dan Ts65Dn?
3. Mengapa K-NN dapat mencapai skor sempurna pada data uji, dan apakah skor tersebut bias?
4. Metode mana yang paling sesuai untuk dataset ini apabila ditinjau dari performa, ketahanan terhadap korelasi antarprotein, dan kemudahan interpretasi?

1.2 Deskripsi Dataset

Dataset yang digunakan adalah *Mice Protein Expression* (berkas `Data_Cortex_Nuclear.xlsx`) yang dipublikasikan oleh Higuera dkk. (2015) dan tersedia di UCI *Machine Learning Repository*. Setiap baris data mewakili satu kali pengukuran ekspresi protein dari sampel korteks seekor tikus. Kolom `MouseID` menyimpan identitas tikus beserta nomor replikasinya, misalnya 309_1 sampai 309_15 untuk tikus bernomor 309. Ringkasan karakteristik dataset disajikan pada Tabel 1.1.

Tabel 1.1 Ringkasan karakteristik dataset

Aspek	Keterangan
Sumber	Higuera, Gardiner, dan Cios (2015), *PLoS ONE*
Jumlah baris	1.095 pengukuran
Jumlah individu tikus	73 ekor (38 Control dan 35 Ts65Dn), masing-masing 15 replikasi pengukuran
Jumlah kolom	82 kolom, terdiri atas 77 fitur protein numerik dan 5 kolom metadata
Kolom metadata	`MouseID`, `Genotype`, `Treatment`, `Behavior`, dan `class`
Variabel target	`Genotype` (Control atau Ts65Dn)
Nilai hilang	Terdapat pada seluruh 77 protein dengan proporsi 0,27% sampai 27,40%

Selain genotipe, tikus dalam penelitian asli juga dibedakan berdasarkan perlakuan obat dan jenis pengondisian perilaku. Perlakuan berupa suntikan Memantine (obat penghambat reseptor NMDA) atau Saline sebagai kontrol, sedangkan perilaku dibedakan menjadi *Context-Shock* (C/S), yaitu tikus yang dirangsang belajar, dan *Shock-Context* (S/C), yaitu tikus yang

tidak dirangsang belajar. Kombinasi ketiganya membentuk kolom `class` dengan delapan kategori, misalnya c-CS-m untuk tikus Control dengan perilaku C/S dan perlakuan Memantine. Distribusi setiap atribut kategorikal ditunjukkan pada Tabel 1.2.

Tabel 1.2 Distribusi atribut kategorikal

Atribut	Kategori	Jumlah	Persentase
`Genotype` (target)	Control	570	52,05%
	Ts65Dn	525	47,95%
`Treatment`	Memantine	585	53,42%
	Saline	510	46,58%
`Behavior`	S/C (*Shock-Context*)	555	50,68%
	C/S (*Context-Shock*)	540	49,32%

Proporsi kelas Control dan Ts65Dn cukup seimbang, yaitu sekitar 52 banding 48. Kondisi ini membuat akurasi tetap layak dipakai sebagai salah satu ukuran performa, meskipun penelitian ini tetap melaporkan *precision*, *recall*, *F1-score*, dan AUC agar penilaian lebih lengkap.

1.3 Tujuan Proyek

Tujuan yang ingin dicapai melalui proyek ini adalah sebagai berikut.

1. Menyusun tahapan praproses yang sesuai untuk data ekspresi protein, meliputi penanganan nilai hilang, pemilihan fitur, normalisasi, dan pembagian data.
2. Membangun dan mengevaluasi model klasifikasi K-NN, *Gaussian Naive Bayes*, dan *Decision Tree* untuk memprediksi genotipe tikus.
3. Menjelaskan penyebab skor sempurna pada K-NN berdasarkan struktur dataset dan cara kerja algoritma, lalu menentukan apakah hasil tersebut bias.
4. Mengidentifikasi protein yang paling membedakan kedua genotipe dan mengaitkannya dengan pengetahuan biologis tentang *Down Syndrome*.
5. Menentukan metode yang paling sesuai untuk dataset ini berdasarkan performa, validitas evaluasi, dan kemudahan interpretasi.

BAB II
METODOLOGI

Penelitian ini dilakukan melalui lima tahap utama, yaitu eksplorasi data, praproses, pemodelan dengan tiga algoritma, evaluasi pada data uji, dan validasi tambahan berbasis individu tikus. Tahap terakhir ditambahkan setelah ditemukan bahwa K-NN memperoleh skor sempurna, sehingga diperlukan pengujian yang lebih ketat untuk memastikan apakah skor tersebut wajar.

2.1 Tahap Praproses Data

2.1.1 Penanganan Nilai Hilang

Seluruh 77 kolom protein memiliki nilai hilang. Jika setiap baris yang mengandung nilai hilang dihapus, sebanyak 543 baris atau hampir setengah dari dataset akan hilang. Karena itu, nilai hilang diisi menggunakan median dari masing-masing kolom protein. Median dipilih karena tidak mudah terpengaruh oleh nilai ekstrem. Beberapa protein memiliki distribusi yang miring ke kanan, sehingga rata-rata akan tertarik ke arah pencilan dan menghasilkan nilai pengganti yang kurang mewakili kondisi umum data.

2.1.2 Pemisahan Fitur dan Pengodean Target

Matriks fitur hanya berisi 77 kolom protein. Kolom `MouseID`, `Treatment`, `Behavior`, dan `class` dikeluarkan dari fitur. Kolom `class` perlu dibuang karena huruf awalnya ("c" atau "t") secara langsung menyatakan genotipe, sehingga mempertahankannya sama saja dengan memberikan jawaban kepada model. Variabel target `Genotype` diubah menjadi bilangan biner, yaitu 0 untuk Control dan 1 untuk Ts65Dn. Ts65Dn ditetapkan sebagai kelas positif karena kelompok inilah yang ingin dideteksi.

2.1.3 Normalisasi Min-Max

Rentang nilai antarprotein sangat berbeda. Sebagai contoh, NR2A_N memiliki nilai maksimum 8,48, sedangkan ARC_N hanya mencapai 0,16. Untuk menyamakan skala, setiap fitur ditransformasikan ke rentang 0 sampai 1 menggunakan normalisasi Min-Max dengan persamaan berikut.

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}} \tag{2.1}$$

dengan:

x : nilai asli suatu protein pada satu sampel

x' : nilai protein setelah dinormalisasi, berada pada rentang 0 sampai 1

$x\text{min}$: nilai terkecil protein tersebut pada seluruh data

$x\text{max}$: nilai terbesar protein tersebut pada seluruh data

Normalisasi ini diterapkan untuk K-NN dan *Gaussian Naive Bayes*. *Decision Tree* dilatih dengan data yang tidak dinormalisasi karena pemisahan pada setiap simpul hanya membandingkan nilai satu fitur dengan suatu ambang batas, sehingga skala fitur tidak memengaruhi hasilnya. Penggunaan data asli juga membuat ambang batas pada pohon lebih mudah ditafsirkan.

2.1.4 Pembagian Data Latih dan Data Uji

Data dibagi menjadi 70% data latih dan 30% data uji menggunakan fungsi `train_test_split` dengan parameter `stratify=y` dan `random_state=42`. Stratifikasi menjaga agar proporsi Control dan Ts65Dn pada kedua bagian sama dengan proporsi pada data keseluruhan, sedangkan `random_state` memastikan hasil pembagian selalu sama setiap kali kode dijalankan. Pembagian ini dilakukan per baris, artinya setiap pengukuran diperlakukan sebagai sampel yang berdiri sendiri. Konsekuensi dari pilihan ini dibahas secara khusus pada Subbab 4.2.3.

2.2 Deskripsi Tiga Metode Klasifikasi

2.2.1 K-Nearest Neighbor (K-NN)

K-NN adalah metode klasifikasi berbasis contoh (*instance-based*) yang tidak membangun model eksplisit pada tahap pelatihan. Algoritma ini hanya menyimpan seluruh data latih, lalu ketika ada sampel baru, K-NN mencari k sampel latih yang jaraknya paling dekat dan menetapkan kelas berdasarkan suara terbanyak (Taunk dkk., 2019; Sarker, 2021). Jarak antara dua sampel dihitung menggunakan jarak Euclidean.

$$d(p,q) = \sqrt{\sum_{j=1}^{n}(p_j - q_j)^2} \tag{2.2}$$

dengan:

$d(p, q)$: jarak Euclidean antara sampel p dan sampel q

pj, qj : nilai protein ke-j (sudah dinormalisasi) pada sampel p dan q

n : jumlah fitur, yaitu 77 protein

Setelah k tetangga terdekat ditemukan, kelas prediksi ditentukan dengan menghitung kelas mana yang paling sering muncul di antara tetangga tersebut.

$$\hat{y} = arg\,max_c \sum_{i=1}^{k} I(y_i = c) \tag{2.3}$$

dengan:

$\hat{y}$: kelas hasil prediksi

c : kandidat kelas, yaitu Control (0) atau Ts65Dn (1)

yi : kelas dari tetangga ke-i

$I(\cdot)$: fungsi indikator yang bernilai 1 jika yi sama dengan c dan bernilai 0 jika tidak

k : jumlah tetangga yang dipertimbangkan

Dalam penelitian ini, nilai k divariasikan sebanyak tujuh kali, yaitu 1, 3, 5, 7, 9, 11, dan 15. Nilai ganjil dipilih agar tidak terjadi hasil seri pada klasifikasi dua kelas.

2.2.2 Gaussian Naive Bayes (GNB)

Gaussian Naive Bayes adalah metode klasifikasi probabilistik yang didasarkan pada Teorema Bayes. Metode ini menghitung peluang suatu sampel termasuk ke dalam setiap kelas, lalu memilih kelas dengan peluang terbesar. Disebut "naive" karena metode ini mengasumsikan bahwa setiap fitur saling bebas apabila kelasnya sudah diketahui (*conditional independence*). Dengan asumsi tersebut, peluang posterior dapat dituliskan sebagai berikut.

$$P(C \mid X) = \frac{P(C) \prod_{j=1}^{n} P(x_j \mid C)}{P(X)} \tag{2.4}$$

dengan:

$P(C|X)$: peluang sampel X termasuk kelas C (peluang posterior)

$P(C)$: peluang awal kelas C, dihitung dari proporsi kelas pada data latih

$P(x$j$|C)$: peluang munculnya nilai protein ke-j apabila kelasnya C (*likelihood*)

$P(X)$: peluang munculnya sampel X, bernilai sama untuk semua kelas sehingga tidak memengaruhi keputusan

Karena fitur protein bersifat kontinu, *likelihood* setiap fitur dihitung dengan asumsi bahwa nilainya mengikuti distribusi normal (Gaussian).

$$P(x_j \mid C) = \frac{1}{\sqrt{2\pi\,\sigma_{jC}^2}}\, e^{-\frac{(x_j - \mu_{jC})^2}{2\sigma_{jC}^2}} \tag{2.5}$$

dengan:

μjC : rata-rata protein ke-*j* pada data latih kelas *C*

σ^2jC : varians protein ke-*j* pada data latih kelas *C*

*x*j : nilai protein ke-*j* pada sampel yang akan diprediksi

Model GNB digunakan dengan parameter bawaan dari pustaka scikit-learn. Asumsi independensi menjadi titik lemah utama metode ini apabila diterapkan pada data protein yang saling berinteraksi.

2.2.3 Decision Tree

Decision Tree membangun struktur pohon dengan cara membagi data secara bertahap. Pada setiap simpul, algoritma memilih satu fitur dan satu ambang batas yang paling baik dalam memisahkan kelas (Charbuty dan Abdulazeez, 2021; Sarker, 2021). Penelitian ini menggunakan kriteria entropi untuk mengukur tingkat ketidakmurnian suatu kelompok data.

$$H(S) = -\sum_{c=1}^{m} p_c \log_2(p_c) \tag{2.6}$$

dengan:

H(*S*) : entropi kelompok data *S*; bernilai 0 jika semua sampel berasal dari satu kelas dan bernilai 1 jika kedua kelas berjumlah sama

*p*c : proporsi sampel kelas *c* di dalam *S*

m : jumlah kelas, yaitu 2

Kualitas suatu pemisahan diukur dengan *Information Gain*, yaitu besarnya penurunan entropi setelah data dibagi berdasarkan fitur tertentu. Fitur dan ambang batas dengan *Information Gain* terbesar dipilih untuk simpul tersebut.

$$IG(S, A) = H(S) - \sum_{v \in V(A)} \frac{|S_v|}{|S|} H(S_v) \tag{2.7}$$

dengan:

IG(*S*, *A*) : *Information Gain* dari pemisahan data *S* menggunakan fitur *A*

V(*A*) : himpunan cabang hasil pemisahan, yaitu cabang "kurang dari atau sama dengan ambang" dan cabang "lebih dari ambang"

*S*v : kelompok data yang masuk ke cabang *v*

|*S*v|, |*S*| : jumlah sampel pada cabang *v* dan jumlah sampel sebelum dipisah

Pohon yang terlalu dalam cenderung menghafal data latih. Untuk mengendalikannya, dua *hyperparameter* dicari nilai terbaiknya menggunakan *GridSearchCV* dengan validasi silang 5 lipatan terstratifikasi dan metrik *F1-score*. Parameter `max_depth` (kedalaman maksimum) diuji pada nilai 2, 3, 4, 5, 6, dan tanpa batas, sedangkan `min_samples_leaf` (jumlah minimum sampel pada daun) diuji pada nilai 1, 5, 10, dan 20.

2.3 Metrik Evaluasi

Evaluasi dilakukan menggunakan matriks konfusi yang membandingkan kelas aktual dengan kelas prediksi. Ts65Dn diperlakukan sebagai kelas positif, sehingga *True Positive* (TP) adalah tikus Ts65Dn yang diprediksi Ts65Dn, *True Negative* (TN) adalah tikus Control yang diprediksi Control, *False Positive* (FP) adalah tikus Control yang salah diprediksi Ts65Dn, dan *False Negative* (FN) adalah tikus Ts65Dn yang salah diprediksi Control. Dari keempat nilai tersebut dihitung metrik berikut.

$$Akurasi = \frac{TP + TN}{TP + TN + FP + FN} \tag{2.8}$$

$$Precision = \frac{TP}{TP + FP} \tag{2.9}$$

$$Recall = \frac{TP}{TP + FN} \tag{2.10}$$

$$F1 - score = 2 \times \frac{Precision \times Recall}{Precision + Recall} \tag{2.11}$$

Akurasi menunjukkan proporsi seluruh prediksi yang benar. *Precision* menunjukkan seberapa banyak prediksi Ts65Dn yang memang benar Ts65Dn, sedangkan *recall* menunjukkan seberapa banyak tikus Ts65Dn yang berhasil dikenali. *F1-score* merupakan rata-rata harmonik dari *precision* dan *recall*, sehingga nilainya hanya tinggi apabila keduanya sama-sama tinggi.

Selain itu, digunakan kurva ROC (*Receiver Operating Characteristic*) yang menggambarkan hubungan antara *True Positive Rate* dan *False Positive Rate* pada berbagai ambang batas peluang. Luas daerah di bawah kurva (AUC) bernilai antara 0,5 untuk model yang setara tebakan acak hingga 1 untuk model yang memisahkan kedua kelas secara sempurna (Tharwat, 2021; Chicco dan Jurman, 2020).

2.4 Validasi Tambahan Berbasis Individu Tikus

Pembagian data per baris memungkinkan replikasi dari tikus yang sama masuk ke data latih dan data uji sekaligus. Untuk mengukur kemampuan model dalam mengenali tikus yang benar-benar baru, dilakukan validasi tambahan menggunakan `StratifiedGroupKFold` dengan 5 lipatan. Identitas tikus diambil dari bagian awal `MouseID` (misalnya 309 dari 309_1) dan digunakan sebagai kelompok, sehingga seluruh 15 replikasi seekor tikus selalu berada pada lipatan yang sama. Normalisasi Min-Max dihitung ulang hanya dari data latih pada setiap lipatan. Pendekatan ini dikenal sebagai validasi berbasis subjek (*subject-wise validation*) dan dianjurkan untuk data dengan pengukuran berulang (Poldrack dkk., 2020; Vandewiele dkk., 2021; Whalen dkk., 2022).

BAB III
IMPLEMENTASI KODE & REPRODUCIBLE

Seluruh eksperimen diimplementasikan dalam bahasa Python menggunakan Jupyter Notebook (`main.ipynb`). Bab ini menjelaskan lingkungan komputasi, struktur proyek, potongan kode utama, dan langkah yang perlu dilakukan untuk mereproduksi hasil penelitian.

3.1 Lingkungan Komputasi

Pustaka dan versi yang digunakan selama eksperimen ditunjukkan pada Tabel 3.1. Daftar pustaka yang sama juga dicatat pada berkas `requirements.txt`.

Tabel 3.1 Pustaka dan versi yang digunakan

Pustaka	Versi	Fungsi
Python	3.12.10	Bahasa pemrograman utama
pandas	3.0.6	Membaca berkas Excel dan mengolah data tabular
numpy	2.5.4	Operasi numerik
openpyxl	3.1.5	Mesin pembaca berkas .xlsx untuk pandas
scikit-learn	1.9.1	Praproses, pemodelan, validasi silang, dan metrik evaluasi
matplotlib	3.11.2	Pembuatan grafik
seaborn	0.13.2	Visualisasi matriks konfusi

3.2 Struktur Proyek

Tabel 3.2 Struktur berkas proyek

Berkas atau Folder	Keterangan
`Data_Cortex_Nuclear.xlsx`	Dataset *Mice Protein Expression*
`main.ipynb`	Notebook berisi seluruh tahapan eksplorasi, praproses, pemodelan, dan evaluasi
`requirements.txt`	Daftar pustaka Python yang dibutuhkan
`README.md`	Dokumentasi ringkas proyek
`Images/`	Folder keluaran grafik beresolusi 300 DPI

Notebook disusun berurutan sehingga setiap sel bergantung pada hasil sel sebelumnya. Urutan isi notebook ditunjukkan pada Tabel 3.3.

Tabel 3.3 Urutan isi notebook main.ipynb

Bagian	Isi
Impor pustaka	Memuat seluruh pustaka dan mengatur gaya grafik
Eksplorasi dan praproses	Membaca data, menghitung distribusi kelas dan nilai hilang, imputasi median, pengodean target, normalisasi, dan pembagian data
K-NN	Melatih K-NN untuk tujuh nilai k, menampilkan tabel metrik dan kurva performa
Gaussian Naive Bayes	Melatih GNB, menghitung metrik, dan menampilkan matriks konfusi
Decision Tree	Mencari *hyperparameter* dengan *GridSearchCV*, memvisualisasikan pohon, mencetak aturan, dan menampilkan 10 protein terpenting
Perbandingan	Menghitung kurva ROC dan AUC ketiga model serta tabel ringkasan akhir

3.3 Potongan Kode Utama

3.3.1 Praproses Data

```
df = pd.read_excel('Data_Cortex_Nuclear.xlsx')
meta_cols = ['MouseID', 'Genotype', 'Treatment', 'Behavior', 'class']
protein_cols = [c for c in df.columns if c not in meta_cols]

# Imputasi median per kolom protein
X_raw = df[protein_cols].copy()
X_imputed = X_raw.fillna(X_raw.median())

# Target biner: Control = 0, Ts65Dn = 1 (kelas positif)
y = (df['Genotype'] == 'Ts65Dn').astype(int)

# Normalisasi Min-Max ke rentang [0, 1]
scaler = MinMaxScaler()
X_scaled = pd.DataFrame(scaler.fit_transform(X_imputed), columns=protein_cols)

# Pembagian 70:30 terstratifikasi
X_train_s, X_test_s, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.30, random_state=42, stratify=y)
X_train_u, X_test_u, _, _ = train_test_split(
    X_imputed, y, test_size=0.30, random_state=42, stratify=y)
```

Variabel berakhiran `_s` berisi data yang sudah dinormalisasi untuk K-NN dan GNB, sedangkan variabel berakhiran `_u` berisi data asli untuk *Decision Tree*. Karena `random_state` dan `stratify` yang digunakan sama, kedua pembagian menghasilkan indeks baris yang identik.

3.3.2 K-Nearest Neighbor

```
k_values = [1, 3, 5, 7, 9, 11, 15]
for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_s, y_train)
    y_pred = knn.predict(X_test_s)
```

```
    # menghitung accuracy, precision, recall, dan F1 untuk kelas Ts65Dn

best_knn = KNeighborsClassifier(n_neighbors=3).fit(X_train_s, y_train)
y_pred_knn = best_knn.predict(X_test_s)
y_prob_knn = best_knn.predict_proba(X_test_s)[:, 1]
```

3.3.3 Gaussian Naive Bayes

```
gnb = GaussianNB()
gnb.fit(X_train_s, y_train)
y_pred_gnb = gnb.predict(X_test_s)
y_prob_gnb = gnb.predict_proba(X_test_s)[:, 1]
cm_gnb = confusion_matrix(y_test, y_pred_gnb)
```

3.3.4 Decision Tree dengan GridSearchCV

```
param_grid = {'max_depth': [2, 3, 4, 5, 6, None],
              'min_samples_leaf': [1, 5, 10, 20]}
cv_strategy = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
dt_base = DecisionTreeClassifier(criterion='entropy', random_state=42)
grid_dt = GridSearchCV(dt_base, param_grid, cv=cv_strategy,
                       scoring='f1', n_jobs=-1, refit=True)
grid_dt.fit(X_train_u, y_train)

best_dt = grid_dt.best_estimator_
y_pred_dt = best_dt.predict(X_test_u)
y_prob_dt = best_dt.predict_proba(X_test_u)[:, 1]
feat_importances = pd.Series(best_dt.feature_importances_, index=protein_cols)
```

3.3.5 Kurva ROC dan AUC

```
fpr_knn, tpr_knn, _ = roc_curve(y_test, y_prob_knn)
roc_auc_knn = auc(fpr_knn, tpr_knn)
fpr_gnb, tpr_gnb, _ = roc_curve(y_test, y_prob_gnb)
roc_auc_gnb = auc(fpr_gnb, tpr_gnb)
fpr_dt, tpr_dt, _ = roc_curve(y_test, y_prob_dt)
roc_auc_dt = auc(fpr_dt, tpr_dt)
```

3.4 Langkah Reproduksi

Hasil pada laporan ini dapat diperoleh kembali dengan langkah-langkah berikut.

1. Unduh atau kloning repositori proyek dari https://github.com/Eruumaa/DATAMINING-MIDTERM.
2. Pastikan Python versi 3.10 atau lebih baru sudah terpasang, kemudian jalankan perintah `pip install -r requirements.txt` pada folder proyek.
3. Buka `main.ipynb` menggunakan Jupyter Notebook, JupyterLab, atau Visual Studio Code.

4. Jalankan seluruh sel secara berurutan melalui menu *Run All*. Pastikan berkas `Data_Cortex_Nuclear.xlsx` berada pada folder yang sama dengan notebook.
5. Tabel metrik akan tampil pada keluaran sel dan seluruh grafik akan tersimpan sebagai berkas PNG.

Semua proses yang melibatkan unsur acak, yaitu pembagian data, validasi silang, dan pembentukan pohon keputusan, menggunakan `random_state=42`. Dengan demikian, angka yang diperoleh akan sama persis setiap kali notebook dijalankan pada versi pustaka yang sama. Kode untuk validasi tambahan berbasis individu tikus (Subbab 2.4) dilampirkan pada Lampiran B.

BAB IV
HASIL & PEMBAHASAN

4.1 EDA (Exploratory Data Analysis)

4.1.1 Ringkasan Dataset

Hasil pembacaan data menunjukkan bahwa dataset terdiri atas 1.095 baris dan 82 kolom, dengan 77 kolom merupakan fitur protein. Setiap identitas tikus pada kolom `MouseID` muncul tepat 15 kali, sehingga 1.095 baris tersebut sebenarnya berasal dari 73 ekor tikus. Fakta ini menjadi kunci dalam memahami hasil K-NN pada Subbab 4.2. Statistik deskriptif sepuluh protein yang paling berpengaruh menurut *Decision Tree* disajikan pada Tabel 4.1. Statistik ini dihitung sebelum imputasi dan normalisasi.

Tabel 4.1 Statistik deskriptif sepuluh protein utama sebelum praproses

Protein	Jumlah	Rata-rata	Median	Min	Maks	Simpangan Baku
APP_N	1.092	0,4055	0,4029	0,2356	0,6327	0,0611
pNR1_N	1.092	0,8188	0,8194	0,2905	1,4082	0,1314
AMPKA_N	1.092	0,3676	0,3577	0,2264	0,7008	0,0626
AcetylH3K9_N	1.080	0,2165	0,1504	0,0525	1,4594	0,1853
BRAF_N	1.092	0,3819	0,3275	0,1439	2,1334	0,2168
S6_N	1.080	0,4292	0,4010	0,1302	0,8226	0,1374
pNUMB_N	1.092	0,3557	0,3467	0,1856	0,6311	0,0637
NR2B_N	1.092	0,5630	0,5622	0,1848	0,9720	0,0898
pPKCAB_N	1.092	1,5260	1,3767	0,5678	3,0614	0,4787
pJNK_N	1.092	0,3132	0,3209	0,0521	0,4934	0,0517

Dari Tabel 4.1 terlihat dua pola penting. Pertama, beberapa protein memiliki distribusi yang miring. Pada AcetylH3K9_N, rata-rata (0,2165) jauh lebih besar dari median (0,1504), dan nilai maksimumnya mencapai 1,4594 atau hampir tujuh kali rata-rata. Pola serupa terjadi pada BRAF_N yang memiliki nilai maksimum 2,1334, padahal mediannya hanya 0,3275. Kedua, skala antarprotein sangat bervariasi. Nilai pPKCAB_N berada di kisaran 0,57 sampai 3,06, sedangkan pJNK_N hanya 0,05 sampai 0,49.

Perbandingan rata-rata ekspresi antara kedua genotipe pada Tabel 4.2 memberikan gambaran awal tentang protein yang membedakan Control dan Ts65Dn.

Tabel 4.2 Rata-rata ekspresi protein utama berdasarkan genotipe

Protein	Control	Ts65Dn	Keterangan
APP_N	0,3746	0,4393	Lebih tinggi pada Ts65Dn
S6_N	0,3847	0,4790	Lebih tinggi pada Ts65Dn
AcetylH3K9_N	0,1738	0,2642	Lebih tinggi pada Ts65Dn
pNR1_N	0,8465	0,7886	Lebih rendah pada Ts65Dn
AMPKA_N	0,3796	0,3544	Lebih rendah pada Ts65Dn
NR2B_N	0,5806	0,5438	Lebih rendah pada Ts65Dn

APP_N rata-rata 17% lebih tinggi pada tikus Ts65Dn. Temuan ini sesuai dengan harapan biologis karena gen APP terletak pada segmen kromosom yang mengalami trisomi, sehingga ekspresinya meningkat akibat efek dosis gen. Meskipun demikian, rentang nilai kedua genotipe masih saling tumpang tindih, sehingga satu protein saja tidak cukup untuk memisahkan kedua kelompok dengan sempurna.

4.1.2 Profil Missing Values dan Justifikasi Imputasi Median

Pemeriksaan nilai hilang menunjukkan bahwa seluruh 77 protein memiliki setidaknya tiga nilai kosong. Total terdapat 1.906 sel kosong dari 84.315 sel protein atau sekitar 2,26%. Meskipun persentasenya kecil, nilai hilang tersebar di 543 baris yang berbeda. Rincian sebarannya ditunjukkan pada Tabel 4.3.

Tabel 4.3 Sebaran nilai hilang pada kolom protein

Protein	Jumlah Nilai Hilang	Persentase
BCL2_N	300	27,40%
H3MeK4_N	285	26,03%
BAD_N	228	20,82%
EGR1_N	225	20,55%
H3AcK18_N	195	17,81%
pCFOS_N	90	8,22%
ELK_N dan Bcatenin_N	18	1,64%
28 protein lainnya	15	1,37%
1 protein lainnya	7	0,64%
40 protein lainnya	3	0,27%

Ada tiga alasan imputasi median dipilih. Pertama, menghapus baris yang mengandung nilai hilang akan membuang 543 dari 1.095 baris atau sekitar 49,6% data, sehingga informasi yang hilang terlalu banyak. Kedua, menghapus kolom dengan nilai hilang tinggi seperti

BCL2_N juga tidak ideal karena protein tersebut tetap terisi pada lebih dari 70% baris dan berpotensi membawa informasi biologis. Ketiga, sebagaimana ditunjukkan pada Subbab 4.1.1, beberapa protein memiliki pencilan. Median tidak terpengaruh oleh pencilan, sehingga nilai yang diisikan lebih mewakili kondisi umum dibandingkan rata-rata.

4.1.3 Rekayasa Fitur dan Pembagian Data

Rekayasa fitur dalam penelitian ini dilakukan dalam bentuk seleksi kolom dan pengodean target. Dari 82 kolom, hanya 77 kolom protein yang digunakan sebagai fitur. Kolom `class` dibuang karena mengandung informasi genotipe secara langsung, sementara `Treatment` dan `Behavior` dibuang karena merupakan kondisi eksperimen dan bukan bagian dari profil protein. Kolom `MouseID` juga dibuang karena hanya berfungsi sebagai identitas.

Pembagian data menghasilkan 766 baris data latih dan 329 baris data uji. Komposisi kelas pada kedua bagian ditunjukkan pada Tabel 4.4.

Tabel 4.4 Komposisi data latih dan data uji

Bagian	Control	Ts65Dn	Total
Data latih (70%)	399	367	766
Data uji (30%)	171	158	329
Total	570	525	1.095

Perlu dicermati bahwa membuang kolom `MouseID` tidak menghapus struktur replikasi di dalam data. Lima belas pengukuran dari tikus yang sama tetap memiliki profil protein yang sangat mirip, hanya saja identitasnya tidak lagi terlihat. Karena pembagian dilakukan per baris, replikasi seekor tikus tersebar ke data latih dan data uji. Hal inilah yang menjadi akar persoalan skor sempurna K-NN.

4.1.4 Urgensi Normalisasi Min-Max untuk K-NN

K-NN menentukan kelas berdasarkan jarak Euclidean pada Persamaan (2.2). Dalam perhitungan jarak, setiap fitur menyumbang selisih kuadrat nilainya. Akibatnya, fitur dengan rentang besar akan mendominasi jarak. Pada dataset ini, rentang NR2A_N mencapai 6,77 dan pCAMKII_N mencapai 6,12, sedangkan rentang ARC_N dan pS6_N hanya sekitar 0,09. Tanpa normalisasi, selisih kecil pada NR2A_N akan jauh lebih berpengaruh daripada selisih besar pada ARC_N, sehingga jarak antar sampel sebagian besar hanya ditentukan oleh segelintir protein bernilai besar.

Normalisasi Min-Max menempatkan seluruh protein pada rentang yang sama, yaitu 0 sampai 1, sehingga setiap protein memiliki kesempatan yang setara dalam menentukan kedekatan antar sampel. GNB juga menggunakan data yang telah dinormalisasi, meskipun metode ini sebenarnya tidak sensitif terhadap skala karena rata-rata dan varians dihitung per fitur. Sementara itu, *Decision Tree* tidak memerlukan normalisasi sama sekali.

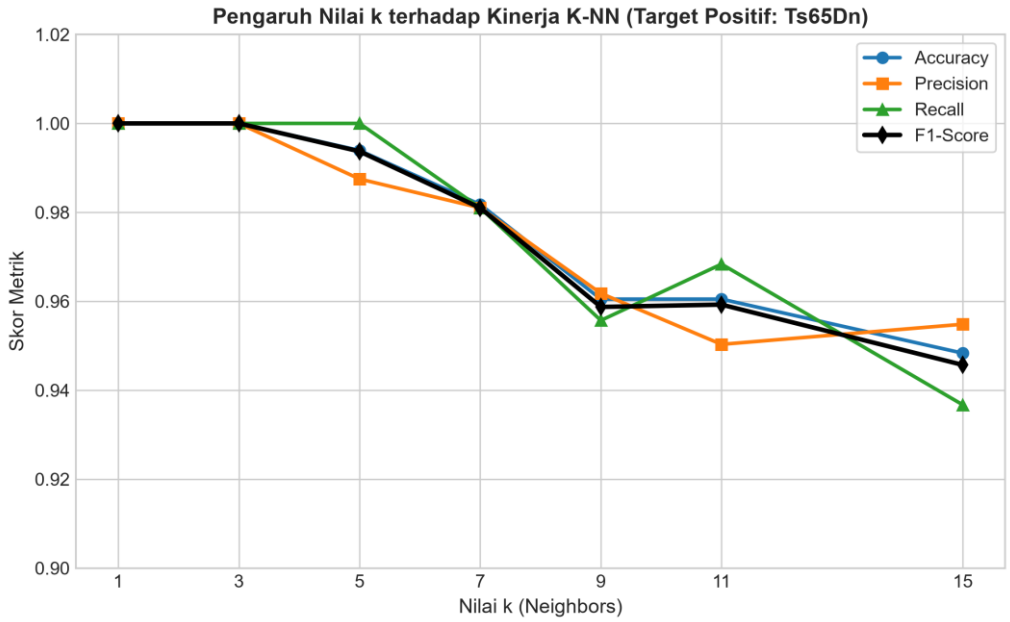
4.2 Klasifikasi dengan K-NN (K-Nearest Neighbor)

4.2.1 Hasil Variasi Nilai k

K-NN dilatih menggunakan data yang telah dinormalisasi dengan tujuh nilai k yang berbeda. Hasil evaluasi pada data uji ditunjukkan pada Tabel 4.5, dengan *precision*, *recall*, dan *F1-score* dihitung untuk kelas Ts65Dn.

Tabel 4.5 Performa K-NN pada data uji untuk berbagai nilai k

k	Akurasi	Precision	Recall	F1-score
1	1,0000	1,0000	1,0000	1,0000
3	1,0000	1,0000	1,0000	1,0000
5	0,9939	0,9875	1,0000	0,9937
7	0,9818	0,9810	0,9810	0,9810
9	0,9605	0,9618	0,9557	0,9587
11	0,9605	0,9503	0,9684	0,9592
15	0,9483	0,9548	0,9367	0,9457



Gambar 4.1 Pengaruh nilai k terhadap performa K-NN

Gambar 4.1 memperlihatkan pola yang jelas. Pada k = 1 dan k = 3, seluruh metrik bernilai 1,0000, artinya tidak ada satu pun dari 329 sampel uji yang salah diklasifikasikan. Ketika k dinaikkan, performa menurun secara bertahap hingga mencapai akurasi 0,9483 dan *F1-score* 0,9457 pada k = 15. Pola ini berbeda dari pola yang biasanya ditemui, yaitu performa meningkat dari k kecil ke k sedang lalu menurun. Pada kasus ini, k terkecil justru memberikan hasil terbaik.

4.2.2 Pemilihan Nilai k Terbaik

Nilai k = 1 dan k = 3 sama-sama menghasilkan *F1-score* 1,0000. Notebook menetapkan k = 3 sebagai konfigurasi terbaik karena keputusan yang diambil dari tiga tetangga lebih tahan terhadap derau dibandingkan keputusan yang hanya bergantung pada satu tetangga. Matriks konfusi K-NN dengan k = 3 menunjukkan TN = 171, FP = 0, FN = 0, dan TP = 158. Nilai AUC yang diperoleh juga 1,0000.

Hasil ini sekilas tampak sangat baik. Namun, skor sempurna pada data uji merupakan hal yang jarang terjadi pada data biologis yang secara alami mengandung variasi antarindividu. Oleh karena itu, hasil ini perlu diklarifikasi sebelum dijadikan dasar kesimpulan.

4.2.3 Klarifikasi Skor Sempurna K-NN dan Penentuan Bias

Subbab ini menjelaskan mengapa K-NN dapat mencapai skor 1,0000 dengan menelusuri dua hal, yaitu bentuk dataset dan cara kerja algoritma K-NN. Penjelasan tersebut kemudian dibuktikan melalui pengujian tambahan.

a. Bentuk dataset: satu tikus diukur 15 kali

Dataset ini bukan kumpulan 1.095 individu yang berbeda, melainkan 73 ekor tikus yang masing-masing diukur 15 kali. Kelima belas pengukuran tersebut berasal dari jaringan otak yang sama, sehingga profil 77 proteinnya sangat mirip satu sama lain. Ketika data dibagi secara acak per baris, setiap tikus menyumbang sebagian replikasinya ke data latih dan sebagian lagi ke data uji. Hasil pemeriksaan menunjukkan bahwa seluruh 73 tikus yang muncul di data uji juga muncul di data latih, dengan jumlah replikasi di data latih antara 6 sampai 14 per tikus atau rata-rata 10,5. Dengan kata lain, tidak ada satu pun sampel uji yang berasal dari tikus yang belum pernah dilihat oleh model.

b. Cara kerja K-NN: mencari sampel yang paling mirip

K-NN tidak mempelajari pola umum yang membedakan Control dan Ts65Dn. Algoritma ini hanya mencari sampel latih yang jaraknya paling dekat, lalu menyalin kelasnya. Apabila di

data latih terdapat replikasi dari tikus yang sama dengan sampel uji, replikasi tersebut hampir pasti menjadi tetangga terdekat. Karena genotipe seekor tikus tidak berubah di antara replikasinya, kelas yang disalin pasti benar. Pada k = 1, mekanisme ini bahkan membuat akurasi pada data latih selalu 1,0000 karena setiap sampel latih menjadi tetangga terdekat bagi dirinya sendiri.

c. Bukti empiris

Untuk membuktikan penjelasan di atas, dilakukan tiga pengujian. Pengujian pertama memeriksa asal tetangga terdekat setiap sampel uji. Hasilnya, pada k = 1 tetangga terdekat dari seluruh 329 sampel uji (100%) berasal dari tikus yang sama. Rata-rata jarak ke replikasi terdekat dari tikus yang sama adalah 0,3881, sedangkan rata-rata jarak ke sampel terdekat dari tikus lain mencapai 0,8220 atau lebih dari dua kali lipatnya. Pada seluruh sampel uji, replikasi dari tikus yang sama selalu lebih dekat dibandingkan sampel mana pun dari tikus lain.

Pengujian kedua membandingkan jarak rata-rata antar sampel. Jarak rata-rata antarreplikasi dalam satu tikus adalah 0,8662. Jarak rata-rata antara tikus berbeda dengan genotipe yang sama adalah 1,6716, sedangkan antara tikus berbeda genotipe adalah 1,7019. Selisih antara dua angka terakhir sangat kecil. Artinya, di ruang 77 dimensi, perbedaan antarindividu jauh lebih menonjol daripada perbedaan antargenotipe. Sampel-sampel mengelompok berdasarkan identitas tikus, bukan berdasarkan genotipenya.

Pengujian ketiga mengulang evaluasi dengan validasi berbasis individu tikus seperti dijelaskan pada Subbab 2.4. Hasilnya dibandingkan dengan pembagian per baris pada Tabel 4.6.

Tabel 4.6 Proporsi tetangga dari tikus yang sama dan akurasi K-NN pada dua skema pembagian data

k	Proporsi Tetangga dari Tikus yang Sama	Akurasi Pembagian per Baris	Akurasi Pembagian per Tikus
1	100,00%	1,0000	0,7128
3	95,64%	1,0000	0,7172
5	85,90%	0,9939	0,7072
7	73,43%	0,9818	0,7110
9	63,12%	0,9605	0,7178
11	55,04%	0,9605	0,7307
15	42,88%	0,9483	0,7271

Tabel 4.6 menjelaskan pola menurun pada Gambar 4.1. Pada k = 1, satu-satunya tetangga selalu berasal dari tikus yang sama. Pada k = 3, sebanyak 95,64% tetangga masih berasal dari tikus yang sama. Semakin besar k, semakin banyak tetangga yang diambil dari tikus lain karena replikasi tikus yang sama di data latih hanya sekitar 10 sampel. Pada k = 15, proporsinya tinggal 42,88%. Begitu K-NN mulai bergantung pada tikus lain, akurasinya ikut turun. Penurunan performa saat k membesar bukan disebabkan oleh k yang terlalu besar, melainkan karena K-NN kehilangan akses ke "kembaran" sampel uji di data latih.

Kolom terakhir Tabel 4.6 memperlihatkan hal yang lebih penting. Ketika seluruh replikasi seekor tikus dipisahkan ke lipatan yang sama, akurasi K-NN turun drastis menjadi 0,71 sampai 0,73 untuk semua nilai k. Nilai k tidak lagi berpengaruh karena K-NN tidak bisa lagi menemukan replikasi tikus yang sama. Angka sekitar 0,72 inilah yang lebih mendekati kemampuan K-NN yang sebenarnya dalam mengenali genotipe pada tikus baru. Pengujian serupa dengan satu kali pembagian 70:30 berbasis tikus (22 tikus pada data uji) juga menghasilkan akurasi K-NN sebesar 0,6970 untuk k = 3, jauh di bawah 1,0000.

d. Penentuan apakah K-NN bias

Berdasarkan ketiga bukti tersebut, skor 1,0000 pada K-NN dapat disimpulkan sebagai hasil yang bias. Bias yang dimaksud di sini adalah bias optimis pada estimasi performa (*optimistic bias*), yaitu kondisi ketika angka evaluasi lebih tinggi dari kemampuan model yang sebenarnya. Sumber biasnya adalah kebocoran informasi pada tingkat individu (*subject-level leakage*), yaitu replikasi dari tikus yang sama muncul di data latih sekaligus data uji (Kapoor dan Narayanan, 2023). Data uji dengan demikian tidak lagi berfungsi sebagai data yang benar-benar baru.

Kondisi ini juga merupakan bentuk *overfitting*. K-NN dengan k kecil pada dasarnya menghafal data latih. Pada dataset ini, yang dihafal adalah "sidik jari" protein setiap tikus, bukan pola yang membedakan genotipe. Selisih antara akurasi 1,0000 pada pembagian per baris dan sekitar 0,72 pada pembagian per tikus adalah celah generalisasi (*generalization gap*) sebesar kurang lebih 28 poin persentase. Celah sebesar ini menunjukkan bahwa model bekerja sangat baik pada individu yang sudah dikenalnya, tetapi tidak demikian pada individu baru.

Istilah bias dalam konteks ini perlu dibedakan dari bias pada konsep *bias-variance trade-off*. Dalam konsep tersebut, K-NN dengan k kecil justru tergolong model berbias rendah tetapi bervarians tinggi, karena batas keputusannya sangat lentur dan mengikuti setiap titik data (Sarker, 2021). Varians tinggi inilah yang membuat K-NN mudah *overfitting*. Jadi, yang bias

bukanlah struktur model K-NN, melainkan angka 1,0000 sebagai perkiraan kemampuan model. Kedua sudut pandang ini mengarah pada kesimpulan yang sama, yaitu skor sempurna tersebut tidak dapat dipercaya sebagai ukuran kemampuan generalisasi.

Catatan Penting Mengenai Skor Sempurna K-NN

Skor 1,0000 yang diperoleh K-NN pada k = 1 dan k = 3 tidak berarti model tersebut sempurna. Pada praktiknya, tidak ada model klasifikasi yang benar-benar sempurna untuk data biologis. Skor tersebut muncul karena 15 replikasi pengukuran dari tikus yang sama tersebar ke data latih dan data uji, sehingga K-NN cukup mencocokkan setiap sampel uji dengan replikasi dari tikus yang sama di data latih.

Dengan demikian, hasil K-NN pada laporan ini dinyatakan **bias secara optimis** dan menunjukkan **overfitting terhadap identitas individu tikus**. Ketika diuji pada tikus yang tidak pernah dilihat saat pelatihan, akurasi K-NN turun menjadi sekitar 71,72%. Angka 1,0000 tetap dilaporkan karena merupakan keluaran dari prosedur yang ditetapkan, tetapi tidak digunakan sebagai dasar untuk menyatakan K-NN sebagai metode terbaik.

4.3 Klasifikasi dengan Gaussian Naive Bayes

4.3.1 Hasil Evaluasi

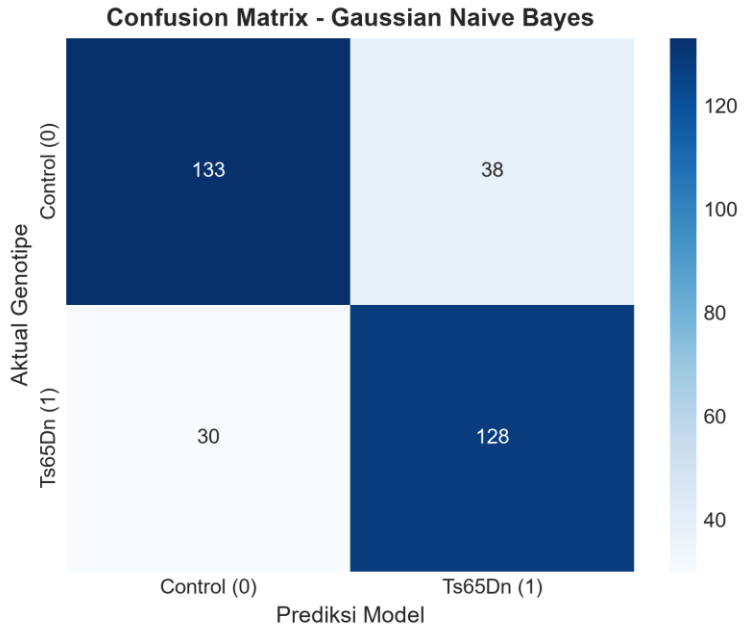
Model GNB dilatih menggunakan data yang telah dinormalisasi. Hasil evaluasinya pada data uji ditunjukkan pada Tabel 4.7 dan Tabel 4.8.

Tabel 4.7 Performa Gaussian Naive Bayes pada data uji

Akurasi	Precision	Recall	F1-score
0,7933	0,7711	0,8101	0,7901

Tabel 4.8 Matriks konfusi Gaussian Naive Bayes

	Prediksi Control	Prediksi Ts65Dn
Aktual Control	TN = 133	FP = 38
Aktual Ts65Dn	FN = 30	TP = 128



Gambar 4.2 Matriks konfusi Gaussian Naive Bayes

GNB memperoleh akurasi 79,33%, yang merupakan nilai terendah di antara ketiga metode. Dari 329 sampel uji, terdapat 68 sampel yang salah diklasifikasikan, terdiri atas 38 tikus Control yang diprediksi sebagai Ts65Dn dan 30 tikus Ts65Dn yang diprediksi sebagai Control. Nilai *recall* (81,01%) sedikit lebih tinggi daripada *precision* (77,11%), yang berarti model cenderung lebih mudah memberi label Ts65Dn. Kecenderungan ini membuat jumlah *False Positive* lebih banyak dibandingkan *False Negative*.

4.3.2 Diskusi Asumsi Conditional Independence

GNB mengasumsikan bahwa setelah genotipe diketahui, ekspresi suatu protein tidak memberikan informasi apa pun tentang ekspresi protein lainnya. Asumsi ini tidak sesuai dengan kenyataan biologis. Protein di korteks bekerja dalam jaringan pensinyalan yang saling terhubung. Contohnya adalah jalur MAPK/ERK, di mana BRAF mengaktifkan MEK yang kemudian mengaktifkan ERK, serta reseptor NMDA yang tersusun atas subunit NR1, NR2A, dan NR2B.

Data pada penelitian ini mengonfirmasi hal tersebut. Dari 2.926 pasangan protein, terdapat 104 pasangan dengan nilai korelasi absolut di atas 0,7 dan 10 pasangan di atas 0,9. Korelasi antara BRAF_N dan pERK_N mencapai 0,928, korelasi antara NR1_N dan NR2A_N sebesar 0,866, dan korelasi antara ERK_N dan NR2A_N sebesar 0,855.

Ketika dua protein berkorelasi tinggi, keduanya membawa informasi yang hampir sama. GNB tetap mengalikan *likelihood* keduanya seolah-olah merupakan bukti yang terpisah,

sehingga informasi yang sama dihitung lebih dari sekali (*double counting*). Akibatnya, peluang posterior menjadi terlalu yakin ke arah salah satu kelas, dan kesalahan klasifikasi meningkat. Meski begitu, Wickramasinghe dan Kalutarage (2021) menunjukkan bahwa *Naive Bayes* masih dapat menghasilkan peringkat peluang yang cukup baik walaupun asumsinya dilanggar. Hal ini tercermin dari nilai AUC GNB sebesar 0,8967 yang relatif tinggi dibandingkan akurasinya. Masalah utamanya terletak pada kalibrasi peluang, bukan sepenuhnya pada kemampuan membedakan kelas.

4.4 Klasifikasi dengan Decision Tree

4.4.1 Hasil Hyperparameter Tuning dan Evaluasi

Pencarian *hyperparameter* dengan *GridSearchCV* menguji 24 kombinasi parameter. Ringkasan hasilnya disajikan pada Tabel 4.9.

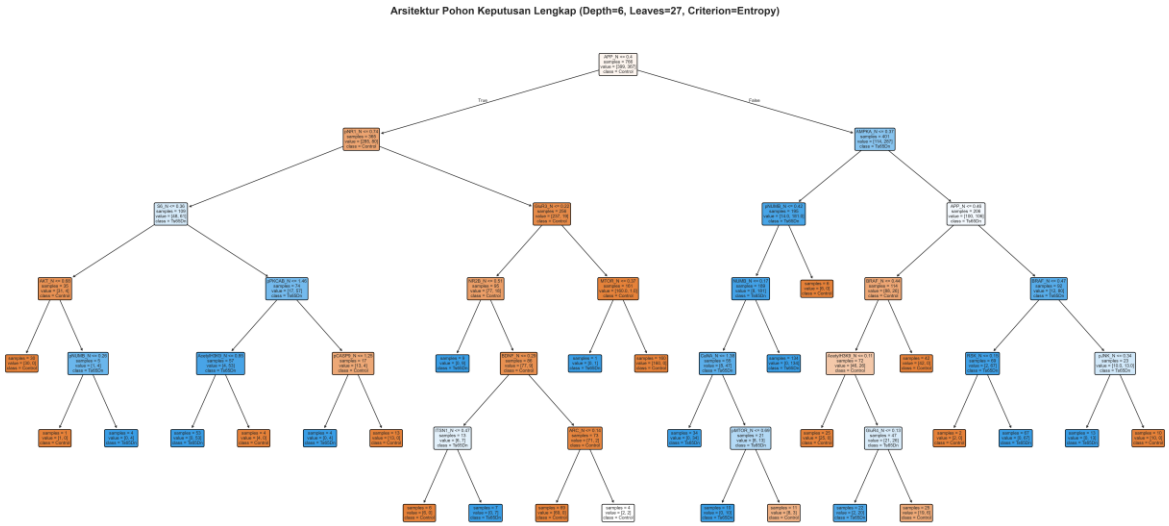
Tabel 4.9 Hasil hyperparameter tuning Decision Tree

Komponen	Nilai
Ruang pencarian `max_depth`	2, 3, 4, 5, 6, dan tanpa batas
Ruang pencarian `min_samples_leaf`	1, 5, 10, dan 20
Parameter terbaik	`max_depth` = 6 dan `min_samples_leaf` = 1
***F1-score* rata-rata validasi silang**	0,8900
Kedalaman pohon dan jumlah daun	6 tingkat dan 27 daun
Jumlah protein yang digunakan pohon	22 dari 77 protein
Akurasi pada data latih	0,9830

Tabel 4.10 Performa Decision Tree pada data uji

Akurasi	Precision	Recall	F1-score	AUC
0,8967	0,9247	0,8544	0,8882	0,9256

Decision Tree memperoleh akurasi 89,67% dengan matriks konfusi TN = 160, FP = 11, FN = 23, dan TP = 135. Berbeda dengan GNB, pohon keputusan lebih berhati-hati dalam memberikan label Ts65Dn. Hal ini terlihat dari *precision* yang tinggi (92,47%) dan *recall* yang sedikit lebih rendah (85,44%). Akurasi pada data latih (98,30%) dan data uji (89,67%) berselisih sekitar 8,6 poin persentase, sehingga masih terdapat sedikit *overfitting*, tetapi jauh lebih terkendali dibandingkan pohon tanpa batas kedalaman. Hanya 22 protein yang dipakai dalam pohon, yang berarti *Decision Tree* sekaligus melakukan seleksi fitur secara otomatis.



Gambar 4.3 Struktur pohon keputusan hasil tuning (kedalaman 6, 27 daun)

4.4.2 Interpretasi Aturan Pohon Keputusan

Gambar 4.3 menunjukkan bahwa simpul akar menggunakan APP_N dengan ambang 0,40. Pemisahan pertama ini sudah membagi data latih menjadi dua kelompok yang cukup berbeda. Cabang kiri (APP_N ≤ 0,40) berisi 365 sampel dengan 285 di antaranya Control, sedangkan cabang kanan (APP_N > 0,40) berisi 401 sampel dengan 287 di antaranya Ts65Dn. Ambang 0,40 ini berada tepat di antara rata-rata APP_N kelompok Control (0,3746) dan Ts65Dn (0,4393) pada Tabel 4.2.

Beberapa aturan menghasilkan daun yang murni dan berisi banyak sampel. Dua contoh di antaranya adalah sebagai berikut.

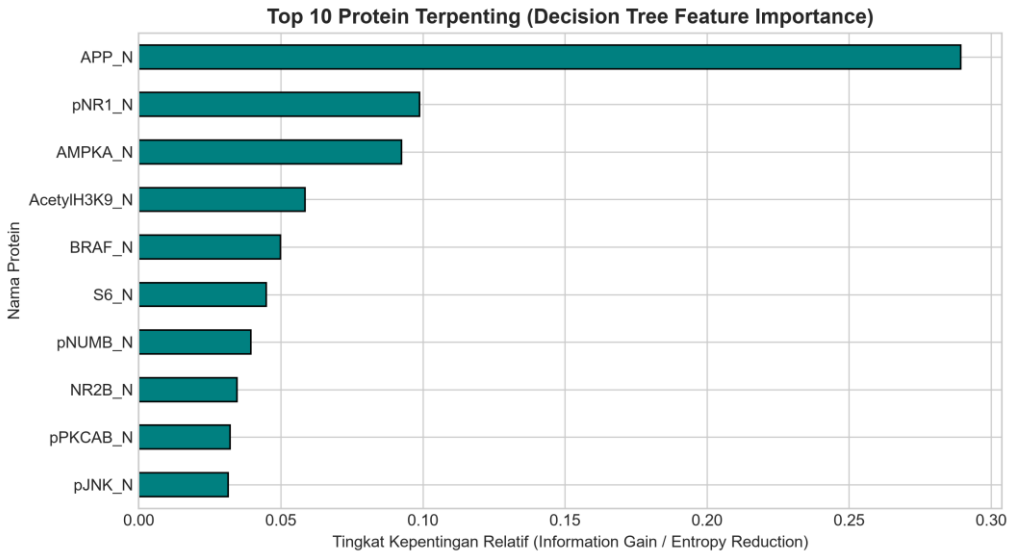
a. Jika APP_N > 0,40, AMPKA_N ≤ 0,37, pNUMB_N ≤ 0,42, dan NUMB_N > 0,17, maka sampel diklasifikasikan sebagai Ts65Dn. Aturan ini mencakup 134 sampel latih tanpa satu pun kesalahan.

b. Jika APP_N ≤ 0,40, pNR1_N > 0,74, GluR3_N > 0,22, dan MTOR_N > 0,37, maka sampel diklasifikasikan sebagai Control. Aturan ini mencakup 160 sampel latih tanpa satu pun kesalahan.

Aturan seperti ini dapat dibaca dan diperiksa langsung oleh peneliti biologi, sesuatu yang tidak dapat diberikan oleh K-NN maupun GNB. Daftar aturan lengkap disajikan pada Lampiran A.

4.4.3 Sepuluh Protein Terpenting dan Kaitan Biologisnya

Tingkat kepentingan setiap protein dihitung dari total penurunan entropi yang dihasilkan protein tersebut di seluruh simpul pohon. Sepuluh protein dengan nilai tertinggi ditunjukkan pada Gambar 4.4 dan Tabel 4.11.



Gambar 4.4 Sepuluh protein terpenting menurut Decision Tree

Tabel 4.11 Sepuluh protein terpenting dan kaitan biologisnya

No	Protein	Kepentingan	Kaitan Biologis
1	APP_N	0,2893	*Amyloid Precursor Protein*. Gennya terletak pada kromosom 21 manusia dan ikut mengalami trisomi pada Ts65Dn. Kelebihan APP berkaitan dengan neuropatologi DS dan Alzheimer dini.
2	pNR1_N	0,0988	Bentuk terfosforilasi subunit NR1 reseptor NMDA yang mengatur plastisitas sinaptik dan *long-term potentiation*.
3	AMPKA_N	0,0925	Kinase sensor energi sel. Perubahannya mencerminkan gangguan metabolisme pada neuron trisomi.
4	AcetylH3K9_N	0,0585	Penanda epigenetik berupa asetilasi histon H3 yang mengatur transkripsi gen terkait memori.
5	BRAF_N	0,0498	Komponen hulu jalur MAPK/ERK yang berperan dalam konsolidasi memori.
6	S6_N	0,0449	Protein ribosom di hilir jalur mTOR yang mengendalikan sintesis protein sinaptik.
7	pNUMB_N	0,0394	Protein yang terlibat dalam diferensiasi sel saraf dan berinteraksi dengan pemrosesan APP.
8	NR2B_N	0,0345	Subunit reseptor NMDA, juga merupakan sasaran kerja obat Memantine.
9	pPKCAB_N	0,0322	*Protein Kinase C* terfosforilasi yang meneruskan sinyal kalsium di sinapsis.
10	pJNK_N	0,0315	Kinase yang terkait dengan respons stres sel dan kematian neuron.

APP_N menyumbang hampir 29% dari total kepentingan, jauh di atas protein lainnya. Temuan ini konsisten dengan biologi DS karena gen APP termasuk dalam segmen yang mengalami trisomi, sehingga ekspresinya naik akibat efek dosis gen. Protein lain di daftar ini sebagian besar berada pada jalur pensinyalan sinaptik, seperti reseptor NMDA (pNR1_N, NR2B_N), jalur MAPK (BRAF_N), dan jalur mTOR (S6_N). Fakta bahwa model berhasil menempatkan protein-protein tersebut di urutan teratas tanpa diberi pengetahuan biologis sebelumnya menunjukkan bahwa pola yang dipelajari *Decision Tree* masuk akal secara biologis.

4.5 Perbandingan dan Evaluasi Model

4.5.1 Tabel Perbandingan Kinerja

Ringkasan performa ketiga metode pada data uji dengan pembagian per baris disajikan pada Tabel 4.12, sedangkan hasil validasi berbasis individu tikus disajikan pada Tabel 4.13.

Tabel 4.12 Perbandingan performa ketiga metode pada data uji (pembagian per baris)

Model	Akurasi	Precision	Recall	F1-score	AUC
K-NN (k = 3)	1,0000	1,0000	1,0000	1,0000	1,0000
Decision Tree	0,8967	0,9247	0,8544	0,8882	0,9256
Gaussian Naive Bayes	0,7933	0,7711	0,8101	0,7901	0,8967

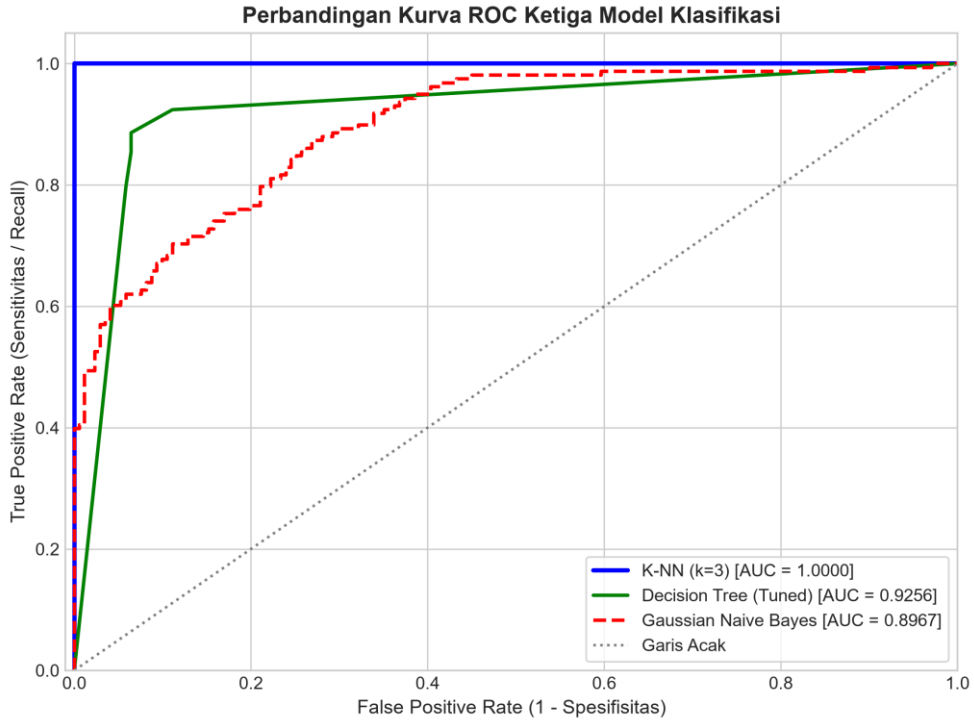
Tabel 4.13 Perbandingan performa pada validasi berbasis individu tikus (StratifiedGroupKFold 5 lipatan)

Model	Akurasi	Precision	Recall	F1-score	Penurunan Akurasi
K-NN (k = 3)	0,7172 ± 0,0981	0,7015	0,6990	0,6927	28,28 poin
Decision Tree	0,8046 ± 0,0660	0,7960	0,8095	0,7968	9,21 poin
Gaussian Naive Bayes	0,6087 ± 0,0906	0,5870	0,6800	0,6143	18,46 poin

Pada Tabel 4.12, K-NN berada di urutan pertama, disusul *Decision Tree* dan GNB. Namun, urutan ini berubah ketika model diuji pada tikus yang tidak pernah dilihat. Pada Tabel 4.13, *Decision Tree* menjadi metode terbaik dengan akurasi rata-rata 80,46%, sementara K-NN turun ke 71,72% dan GNB ke 60,87%. Penurunan akurasi *Decision Tree* hanya sekitar 9 poin, jauh lebih kecil dibandingkan K-NN yang turun lebih dari 28 poin. Ketiga model mengalami penurunan, yang menegaskan bahwa kebocoran replikasi menguntungkan semua metode. Meski begitu, K-NN paling diuntungkan karena cara kerjanya yang mencari sampel paling mirip.

Simpangan baku pada Tabel 4.13 cukup besar, yaitu 0,07 sampai 0,10. Hal ini wajar karena setiap lipatan hanya berisi sekitar 15 tikus pada data uji, sehingga hasil sangat dipengaruhi oleh tikus mana yang kebetulan masuk ke lipatan tersebut. Pada satu kali pembagian 70:30 berbasis tikus, akurasi K-NN, *Decision Tree*, dan GNB masing-masing sebesar 0,6970, 0,6606, dan 0,6455. Artinya, keunggulan *Decision Tree* dalam rata-rata validasi silang tidak selalu muncul pada setiap pembagian tunggal, tetapi ketiga model secara konsisten berada jauh di bawah skor yang diperoleh pada pembagian per baris.

4.5.2 Karakteristik Kurva ROC dan Nilai AUC



Gambar 4.5 Perbandingan kurva ROC ketiga model

Gambar 4.5 menunjukkan kurva ROC ketiga model pada data uji. Kurva K-NN langsung naik ke titik (0, 1) dan menghasilkan AUC 1,0000. Bentuk ini adalah konsekuensi dari bias yang telah dijelaskan pada Subbab 4.2.3. Ketiga tetangga terdekat hampir selalu berasal dari tikus yang sama, sehingga peluang yang dihasilkan hampir selalu 0 atau 1 dan kedua kelas terpisah sempurna.

Kurva *Decision Tree* berbentuk patah-patah dengan sedikit titik sudut karena pohon hanya memiliki 27 daun, sehingga jumlah nilai peluang yang dapat dihasilkan terbatas. Meskipun begitu, kurvanya naik tajam di bagian awal dan mencapai AUC 0,9256. Kurva GNB lebih halus karena peluangnya bersifat kontinu, dengan AUC 0,8967. Pada *False Positive Rate* yang sangat rendah, GNB bahkan mampu menyamai *Decision Tree*, tetapi tertinggal di wilayah

tengah kurva. Hal ini sejalan dengan pembahasan pada Subbab 4.3.2 bahwa GNB cukup baik dalam mengurutkan sampel, tetapi peluangnya kurang terkalibrasi.

4.5.3 Analisis Kesesuaian Metode untuk Dataset Mice Protein Expression

Kesesuaian setiap metode dinilai berdasarkan empat aspek, yaitu jumlah data dibandingkan jumlah fitur, korelasi antarprotein, validitas evaluasi, dan kebutuhan interpretasi. Ringkasannya disajikan pada Tabel 4.14.

Tabel 4.14 Analisis kesesuaian metode terhadap karakteristik dataset

Aspek	Temuan dan Dampaknya
Jumlah data dibandingkan jumlah fitur	Secara baris, rasionya sekitar 14 sampel per fitur (1.095 banding 77). Namun, jumlah individu yang benar-benar independen hanya 73 ekor, lebih sedikit dari jumlah fitur. Kondisi ini membuat model yang terlalu lentur mudah menghafal individu. *Decision Tree* dengan batas kedalaman dan seleksi fitur otomatis lebih tahan terhadap masalah ini.
Korelasi antarprotein	Terdapat 104 pasangan protein dengan korelasi absolut di atas 0,7. Korelasi ini melanggar asumsi GNB dan menurunkan performanya. *Decision Tree* tidak terganggu karena hanya memilih satu fitur terbaik di setiap simpul, sedangkan K-NN tidak terpengaruh secara langsung oleh korelasi.
Validitas evaluasi	Struktur replikasi membuat skor K-NN bias optimis. *Decision Tree* mengalami penurunan paling kecil ketika diuji pada tikus baru, sehingga hasilnya paling dapat dipercaya.
Kebutuhan interpretasi	Dalam riset biomedis, peneliti perlu mengetahui protein mana yang berperan. Hanya *Decision Tree* yang menyediakan aturan eksplisit dan peringkat kepentingan protein, dengan APP_N sebagai penentu utama.

Berdasarkan keempat aspek tersebut, *Decision Tree* merupakan metode yang paling sesuai untuk dataset ini. K-NN memang unggul pada pembagian per baris, tetapi keunggulan tersebut tidak bertahan ketika diuji pada individu baru. GNB paling tidak sesuai karena asumsi dasarnya bertentangan dengan sifat data protein.

4.5.4 Analisis Komparatif Kelebihan dan Kekurangan Metode

Tabel 4.15 Kelebihan dan kekurangan ketiga metode

Metode	Kelebihan	Kekurangan
K-NN	Sederhana dan tidak memerlukan asumsi distribusi. Sangat efektif mengenali sampel yang mirip dengan data latih. Mencapai skor tertinggi pada pembagian per baris.	Wajib dinormalisasi. Mudah *overfitting* pada k kecil dan paling sensitif terhadap kebocoran replikasi. Tidak menghasilkan aturan atau peringkat fitur. Proses prediksi lambat untuk data besar karena harus menghitung jarak ke seluruh data latih.
Gaussian Naive Bayes	Pelatihan sangat cepat dan tidak memerlukan *hyperparameter tuning*. Masih menghasilkan AUC yang cukup baik (0,8967).	Asumsi independensi dilanggar oleh korelasi antarprotein. Menghasilkan kesalahan klasifikasi terbanyak (68 sampel) dan performa terendah pada kedua skema evaluasi.
Decision Tree	Aturan *IF-THEN* mudah dibaca. Tidak memerlukan normalisasi. Tahan terhadap korelasi antarfitur dan melakukan seleksi fitur otomatis. Paling stabil ketika diuji pada tikus baru.	Rentan *overfitting* jika kedalaman tidak dibatasi. Struktur pohon dapat berubah cukup banyak apabila data latih sedikit berubah. Kurva ROC kurang halus karena jumlah daun terbatas.

BAB V
KESIMPULAN DAN REKOMENDASI

5.1 Kesimpulan

Berdasarkan hasil eksperimen dan pembahasan, dapat ditarik kesimpulan sebagai berikut.

1. Praproses yang terdiri atas imputasi median, pembuangan kolom metadata, pengodean target biner, dan normalisasi Min-Max berhasil menyiapkan data yang layak untuk ketiga metode. Imputasi median dipilih karena dapat mempertahankan 543 baris yang mengandung nilai hilang dan tidak terpengaruh oleh pencilan.
2. Pada pembagian data per baris, K-NN dengan k = 3 memperoleh akurasi, *F1-score*, dan AUC sebesar 1,0000, diikuti oleh *Decision Tree* (akurasi 89,67%, AUC 0,9256) dan GNB (akurasi 79,33%, AUC 0,8967).
3. Skor sempurna K-NN terbukti bias secara optimis. Penyebabnya adalah struktur dataset yang berisi 15 replikasi per tikus sehingga seluruh tikus pada data uji juga ada di data latih, dikombinasikan dengan cara kerja K-NN yang mencari sampel paling mirip. Tetangga terdekat dari 100% sampel uji berasal dari tikus yang sama. Ketika diuji pada tikus baru, akurasi K-NN turun menjadi 71,72%. K-NN dengan demikian mengalami *overfitting* terhadap identitas individu, bukan mempelajari pola genotipe.
4. GNB memberikan performa terendah karena asumsi independensi bersyarat dilanggar oleh korelasi tinggi antarprotein yang berada pada jalur pensinyalan yang sama.
5. *Decision Tree* merupakan metode yang paling sesuai untuk dataset ini. Metode ini memiliki performa yang baik, paling stabil ketika diuji pada tikus baru (akurasi 80,46%), dan menghasilkan aturan yang dapat ditafsirkan. APP_N teridentifikasi sebagai protein paling berpengaruh, sejalan dengan letak gen APP pada segmen kromosom yang mengalami trisomi.

5.2 Rekomendasi

Adapun rekomendasi yang dapat dilakukan kedepannya yaitu:

1. Penelitian selanjutnya sebaiknya menggunakan pembagian data berbasis individu tikus, misalnya `GroupKFold`, `StratifiedGroupKFold`, atau *Leave-One-Mouse-Out*, agar performa yang dilaporkan mencerminkan kemampuan model pada individu baru.
2. Alternatif lain adalah merata-ratakan 15 replikasi setiap tikus menjadi satu profil sebelum pemodelan, sehingga setiap baris benar-benar mewakili satu individu.
3. Proses normalisasi sebaiknya dihitung hanya dari data latih, lalu diterapkan ke data uji, agar tidak ada informasi dari data uji yang ikut memengaruhi pelatihan.
4. Metode ansambel berbasis pohon seperti *Random Forest* atau *Gradient Boosting* layak dicoba untuk meningkatkan performa sambil tetap mempertahankan informasi kepentingan fitur.
5. Untuk penerapan di bidang biomedis, *Decision Tree* dapat digunakan sebagai alat bantu penapisan biomarker karena aturan dan ambang batas proteinnya dapat diverifikasi secara laboratorium.

REFERENSI

Antonarakis, S. E., Skotko, B. G., Rafii, M. S., Strydom, A., Pape, S. E., Bianchi, D. W., Sherman, S. L., & Reeves, R. H. (2020). Down syndrome. *Nature Reviews Disease Primers, 6*(1), 9. https://doi.org/10.1038/s41572-019-0143-7

Charbuty, B., & Abdulazeez, A. (2021). *Classification based on decision tree algorithm for machine learning*. *Journal of Applied Science and Technology Trends, 2*(01), 20-28. https://doi.org/10.38094/jastt20165

Chicco, D., & Jurman, G. (2020). *The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation*. *BMC Genomics, 21*(1), 6. https://doi.org/10.1186/s12864-019-6413-7

Contestabile, A., Magara, S., & Cancedda, L. (2017). *The GABAergic hypothesis for cognitive disabilities in Down syndrome*. *Frontiers in Cellular Neuroscience, 11*, 54. https://doi.org/10.3389/fncel.2017.00054

Herault, Y., Delabar, J. M., Fisher, E. M. C., Tybulewicz, V. L. J., Yu, E., & Brault, V. (2017). *Rodent models in Down syndrome research: Impact and future opportunities*. *Disease Models & Mechanisms, 10*(10), 1165-1186. https://doi.org/10.1242/dmm.029728

Higuera, C., Gardiner, K. J., & Cios, K. J. (2015). *Self-organizing feature maps identify proteins critical to learning in a mouse model of Down syndrome*. *PLoS ONE, 10*(6), e0129126. https://doi.org/10.1371/journal.pone.0129126

Kapoor, S., & Narayanan, A. (2023). *Leakage and the reproducibility crisis in machine-learning-based science*. *Patterns, 4*(9), 100804. https://doi.org/10.1016/j.patter.2023.100804

Poldrack, R. A., Huckins, G., & Varoquaux, G. (2020*). Establishment of best practices for evidence for prediction: A review*. *JAMA Psychiatry, 77*(5), 534-540. https://doi.org/10.1001/jamapsychiatry.2019.3671

Sarker, I. H. (2021). *Machine learning: Algorithms, real-world applications and research directions*. *SN Computer Science, 2*(3), 160. https://doi.org/10.1007/s42979-021-00592-x

Taunk, K., De, S., Verma, S., & Swetapadma, A. (2019). *A brief review of nearest neighbor algorithm for learning and classification*. In *2019 International Conference on Intelligent Computing and Control Systems (ICCS)* (pp. 1255-1260). IEEE. https://doi.org/10.1109/ICCS45141.2019.9065747

Tharwat, A. (2021). *Classification assessment methods*. *Applied Computing and Informatics, 17*(1), 168-192. https://doi.org/10.1016/j.aci.2018.08.003

Vandewiele, G., Dehaene, I., Kovacs, G., Sterckx, L., Janssens, O., Ongenae, F., De Backere, F., & De Turck, F. (2021). *Overly optimistic prediction results on imbalanced data: A case study of flaws and benefits when applying over-sampling*. *Artificial Intelligence in Medicine, 111*, 101987. https://doi.org/10.1016/j.artmed.2020.101987

Whalen, S., Schreiber, J., Noble, W. S., & Pollard, K. S. (2022). *Navigating the pitfalls of applying machine learning in genomics*. *Nature Reviews Genetics, 23*(3), 169-181. https://doi.org/10.1038/s41576-021-00434-9

Wickramasinghe, I., & Kalutarage, H. (2021). *Naive Bayes: Applications, variations and vulnerabilities: A review of literature with code snippets for implementation*. *Soft Computing, 25*(3), 2277-2293. https://doi.org/10.1007/s00500-020-05297-6

LAMPIRAN

Lampiran A. Aturan Lengkap Pohon Keputusan

Aturan berikut dihasilkan oleh fungsi `export_text` dari model *Decision Tree* terbaik. Angka pada bagian *weights* menunjukkan jumlah sampel latih Control dan Ts65Dn pada daun tersebut, sedangkan *class* 0 berarti Control dan *class* 1 berarti Ts65Dn.

```
|--- APP_N <= 0.40
|   |--- pNR1_N <= 0.74
|   |   |--- S6_N <= 0.36
|   |   |   |--- AKT_N <= 0.68
|   |   |   |   |--- weights: [30.00, 0.00] class: 0
|   |   |   |--- AKT_N >  0.68
|   |   |   |   |--- pNUMB_N <= 0.26
|   |   |   |   |   |--- weights: [1.00, 0.00] class: 0
|   |   |   |   |--- pNUMB_N >  0.26
|   |   |   |   |   |--- weights: [0.00, 4.00] class: 1
|   |   |--- S6_N >  0.36
|   |   |   |--- pPKCAB_N <= 1.46
|   |   |   |   |--- AcetylH3K9_N <= 0.85
|   |   |   |   |   |--- weights: [0.00, 53.00] class: 1
|   |   |   |   |--- AcetylH3K9_N >  0.85
|   |   |   |   |   |--- weights: [4.00, 0.00] class: 0
|   |   |   |--- pPKCAB_N >  1.46
|   |   |   |   |--- pCASP9_N <= 1.25
|   |   |   |   |   |--- weights: [0.00, 4.00] class: 1
|   |   |   |   |--- pCASP9_N >  1.25
|   |   |   |   |   |--- weights: [13.00, 0.00] class: 0
|   |--- pNR1_N >  0.74
|   |   |--- GluR3_N <= 0.22
|   |   |   |--- NR2B_N <= 0.51
|   |   |   |   |--- weights: [0.00, 9.00] class: 1
|   |   |   |--- NR2B_N >  0.51
|   |   |   |   |--- BDNF_N <= 0.29
|   |   |   |   |   |--- ITSN1_N <= 0.47
|   |   |   |   |   |   |--- weights: [6.00, 0.00] class: 0
|   |   |   |   |   |--- ITSN1_N >  0.47
|   |   |   |   |   |   |--- weights: [0.00, 7.00] class: 1
|   |   |   |   |--- BDNF_N >  0.29
|   |   |   |   |   |--- ARC_N <= 0.14
|   |   |   |   |   |   |--- weights: [69.00, 0.00] class: 0
|   |   |   |   |   |--- ARC_N >  0.14
|   |   |   |   |   |   |--- weights: [2.00, 2.00] class: 0
|   |   |--- GluR3_N >  0.22
|   |   |   |--- MTOR_N <= 0.37
|   |   |   |   |--- weights: [0.00, 1.00] class: 1
|   |   |   |--- MTOR_N >  0.37
|   |   |   |   |--- weights: [160.00, 0.00] class: 0
|--- APP_N >  0.40
|   |--- AMPKA_N <= 0.37
|   |   |--- pNUMB_N <= 0.42
|   |   |   |--- NUMB_N <= 0.17
|   |   |   |   |--- CaNA_N <= 1.38
|   |   |   |   |   |--- weights: [0.00, 34.00] class: 1
|   |   |   |   |--- CaNA_N >  1.38
|   |   |   |   |   |--- pMTOR_N <= 0.69
|   |   |   |   |   |   |--- weights: [0.00, 10.00] class: 1
|   |   |   |   |   |--- pMTOR_N >  0.69
|   |   |   |   |   |   |--- weights: [8.00, 3.00] class: 0
|   |   |   |--- NUMB_N >  0.17
|   |   |   |   |--- weights: [0.00, 134.00] class: 1
|   |   |--- pNUMB_N >  0.42
|   |   |   |--- weights: [6.00, 0.00] class: 0
|   |--- AMPKA_N >  0.37
|   |   |--- APP_N <= 0.46
|   |   |   |--- BRAF_N <= 0.44
```

```
|   |   |   |   |--- AcetylH3K9_N <= 0.11
|   |   |   |   |   |--- weights: [25.00, 0.00] class: 0
|   |   |   |   |--- AcetylH3K9_N >  0.11
|   |   |   |   |   |--- GluR4_N <= 0.13
|   |   |   |   |   |   |--- weights: [2.00, 20.00] class: 1
|   |   |   |   |   |--- GluR4_N >  0.13
|   |   |   |   |   |   |--- weights: [19.00, 6.00] class: 0
|   |   |   |--- BRAF_N >  0.44
|   |   |   |   |--- weights: [42.00, 0.00] class: 0
|   |   |--- APP_N >  0.46
|   |   |   |--- BRAF_N <= 0.47
|   |   |   |   |--- RSK_N <= 0.15
|   |   |   |   |   |--- weights: [2.00, 0.00] class: 0
|   |   |   |   |--- RSK_N >  0.15
|   |   |   |   |   |--- weights: [0.00, 67.00] class: 1
|   |   |   |--- BRAF_N >  0.47
|   |   |   |   |--- pJNK_N <= 0.34
|   |   |   |   |   |--- weights: [0.00, 13.00] class: 1
|   |   |   |   |--- pJNK_N >  0.34
|   |   |   |   |   |--- weights: [10.00, 0.00] class: 0
```

Lampiran B. Kode Validasi Berbasis Individu Tikus

Kode berikut digunakan untuk menghasilkan Tabel 4.6 dan Tabel 4.13. Kode ini dijalankan setelah sel praproses pada `main.ipynb`, sehingga variabel `df`, `protein_cols`, `X_imputed`, `y`, dan data hasil pembagian sudah tersedia.

```
import numpy as np
from sklearn.model_selection import StratifiedGroupKFold
from sklearn.neighbors import NearestNeighbors

# Identitas tikus diambil dari bagian awal MouseID, contoh: '309_1' menjadi '309'
mouse = df['MouseID'].str.rsplit('_', n=1).str[0].values
m_train = mouse[X_train_s.index]
m_test = mouse[X_test_s.index]
print('Tikus di data uji yang juga ada di data latih:',
      len(set(m_test) & set(m_train)), 'dari', len(set(m_test)))

# 1) Proporsi tetangga terdekat yang berasal dari tikus yang sama
nn = NearestNeighbors(n_neighbors=15).fit(X_train_s.values)
_, idx = nn.kneighbors(X_test_s.values)
for k in [1, 3, 5, 7, 9, 11, 15]:
    same = (m_train[idx[:, :k]] == m_test[:, None]).mean()
    print(f'k={k}: proporsi tetangga dari tikus yang sama = {same:.4f}')

# 2) Validasi berbasis individu tikus (5 lipatan)
def group_cv(make_model, scale=True):
    sgkf = StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=42)
    X, t = X_imputed.values, y.values
    acc = []
    for tr, te in sgkf.split(X, t, groups=mouse):
        Xa, Xb = X[tr], X[te]
        if scale:  # scaler hanya di-fit pada data latih lipatan
            sc = MinMaxScaler().fit(Xa)
            Xa, Xb = sc.transform(Xa), sc.transform(Xb)
        pred = make_model().fit(Xa, t[tr]).predict(Xb)
        acc.append(accuracy_score(t[te], pred))
    return np.mean(acc), np.std(acc)

for k in [1, 3, 5, 7, 9, 11, 15]:
    print('K-NN k =', k, group_cv(lambda: KNeighborsClassifier(n_neighbors=k)))
```

```
print('GNB', group_cv(GaussianNB))
print('DT ', group_cv(lambda: DecisionTreeClassifier(
    criterion='entropy', max_depth=6, min_samples_leaf=1, random_state=42),
scale=False))
```

Lampiran C. Source Code

https://github.com/Eruumaa/DATAMINING-MIDTERM